prisma-schema-router: Build Spec

Draft v0.1 · October 23, 2026 · Working name (alternative: prisma-dynamic-tenancy)

1. The pitch

The operations layer for schema-per-tenant PostgreSQL on Prisma: route, guard, migrate, govern, verify.

Routing a Prisma client to a tenant's schema is the easy 10%. Prisma's pg adapter already accepts a `schema` option, so a shared pool plus AsyncLocalStorage gets you a working router in an afternoon. What hurts once you have hundreds of tenants is everything around it: applying migrations to every schema safely, proving no query can land in the wrong schema, stopping one busy tenant from starving the rest, and keeping memory and connections bounded.

Each of those pieces exists somewhere (blog posts, example repos, framework-specific modules, commercial tools). No maintained package I found ships them together, and none offers the piece that builds the most trust: an **isolation verification suite you can run in CI**. That combination is the product. A bare router would be a weekend of code that others can copy; the operations layer is the part worth maintaining.

2. Real engineering problems it solves

1. **Connection multiplication.** A pool per tenant exhausts Postgres connection limits, especially on managed Postgres. Connections must stay bounded no matter how many tenants exist.
2. **Isolation leaks.** The realistic leak vectors are unqualified raw SQL, lost async context, background jobs with no tenant, scripts importing a global client, and a cached client bound to the wrong schema.
3. **Fleet migrations.** Prisma Migrate targets one schema at a time. With N tenants you need ordering, concurrency limits, partial-failure handling, resumability, version tracking, and a plan for app and schema version skew.
4. **Unbounded client cache.** A client per tenant cached forever grows with tenant count. Prisma's guidance for long-lived servers is to reuse a client, so a per-tenant cache needs eviction and clean disposal.
5. **Noisy neighbor.** A small shared pool lets one tenant's slow queries block everyone else.

6. **Lifecycle.** Provisioning that can half-fail, deprovisioning that destroys data, and drift between the template schema and reality.

3. Goals and non-goals

Goals

- Bounded connections and memory at 10, 1,000 and 5,000 tenants.
- Fail-closed isolation: a missing or wrong tenant context produces an error, never a query against `public`.
- Safe, resumable migrations across the whole fleet.
- Evidence over claims: published benchmarks and a runnable verification suite.
- Framework-free core with thin adapters.

Non-goals (v1)

- Replacing Prisma or wrapping its query API.
- Replacing row-level security. Shared schema plus a tenant column with RLS is the better default for many products, and the docs should say so.
- MySQL or other databases (their “schema” means something different).
- Cross-region sharding.
- Guaranteeing that application code is bug-free. The library reduces risk, it does not prove absence of bugs.

4. Differentiators (what to build that others don’t)

#	Feature	Why it matters
D1	Fleet migration engine	The biggest unsolved pain; most guides stop at routing
D2	Layered fail-closed guard	Turns “we think it’s isolated” into enforced behavior
D3	Isolation conformance suite (`psr verify`)	Produces evidence users can run in CI and show auditors
D4	Pool governor	Fair sharing of a small pool across tenants

#	Feature	Why it matters
D5	Tiered placement (later)	Move a big tenant to a dedicated pool or database behind the same API
D6	Observability	Tenant-aware metrics and traces out of the box

5. Architecture

Two planes. The **control plane** is a small control database holding the tenant registry and migration state. The **data plane** is the shared pool serving tenant schemas.

```
request / job
   |  resolve tenant (subdomain | header | JWT | signed job claim)
   ▼
registry lookup (control DB, cached, status check)
   ▼
router.run(tenant, fn) ── AsyncLocalStorage ──▶ router.db()
   |                                                |
   ▼                                                ▼
governor (per-tenant concurrency)          client cache (LRU) ─▶
adapter(schema)
                                                        |
                                              shared pg.Pool (bounded)
```

Packages (monorepo)

Package	Responsibility
`@psr/core`	Router, context, client cache, guard, registry interface
`@psr/migrate`	Fleet migration engine and CLI
`@psr/provision`	Tenant lifecycle: create, suspend, archive, delete
`@psr/governor`	Per-tenant concurrency and fair queueing (can be bundled in core)
`@psr/testing`	Isolation conformance suite, ephemeral tenants
`@psr/otel`	Metrics and tracing

Package	Responsibility
`@psr/adapters-*`	Express, Fastify, NestJS, Next.js, SQS, BullMQ

Keep `@psr/core` dependency-light: `pg`, `@prisma/adapter-pg` and Prisma as peer dependencies only.

6. Component design

6.1 Core router

- **Schema naming.** Schema names are generated by the system, for example `t_<slug>_<8 hex chars>`, stored in the registry, and never derived from request input. Validate against a strict pattern and quote identifiers anywhere they appear in SQL.
- **Context.** `router.run(tenant, fn)` stores `{ tenant, db }` in AsyncLocalStorage. `router.db()` reads it and throws `TenantContextError` if nothing is set.
- **Client cache.** Single-flight creation (a map of schema to promise so concurrent first requests build one client), LRU with `maxClients`, an idle TTL, and disposal on eviction.
- **Shared pool safety.** Verify that disconnecting an evicted client does not close the shared pool. If it does, wrap the pool so `end()` is a no-op for tenant clients and only the router can close it.
- **Listener growth.** The current code calls `setMaxListeners(0)`, which only silences Node's warning. Find what adds listeners (likely one per adapter or client creation), attach a single pool-level error handler instead, and keep a regression test asserting listener count stays flat across 1,000 tenant switches.
- **Tenant resolution.** Pluggable resolvers: subdomain, header, JWT claim, custom function. Resolution returns an id only; schema comes from the registry.

6.2 Guard (defense in depth)

Layer	Mechanism
L1 Context	`db()` throws outside `run()`. Ship an ESLint rule that bans importing a global Prisma client. Control-plane access goes through an explicit, logged `router.system()` call.

Layer	Mechanism
L2 Connection default	On each new pooled connection, set `search_path` to an empty guard schema (`psr_guard`). Prisma model queries are schema-qualified by the adapter, so they are unaffected (verify per Prisma version). Unqualified raw SQL errors instead of silently reading `public` .
L3 Raw SQL	`router.raw()` runs inside a transaction with `SET LOCAL search_path` to the tenant schema. `SET LOCAL` ends with the transaction, so it stays safe under transaction-mode poolers.
L4 Roles	Separate migration role (DDL) and runtime role (no CREATE, no access to unrelated schemas). Optional, documented hardening.
L5 Status gate	Suspended, archived or mid-migration tenants are rejected or made read-only at resolve time.

Modes: `strict` (default), `warn` (log only, for adoption), `off` .

6.3 Fleet migrations (D1)

Source of truth: the app's normal `prisma/migrations` folder. Users keep writing migrations the usual way.

Engine v1 (ship first). Orchestrate the official `prisma migrate deploy` once per schema, setting the target schema per invocation (verify the exact mechanism for your Prisma version). Correctness over speed.

Engine v2 (later, opt-in). A native applier that runs each `migration.sql` inside a transaction with `SET LOCAL search_path` , writing rows compatible with Prisma's migration table, so users can still use Prisma's own tooling. Ship only after a compatibility test suite passes.

Orchestration

- `plan` : list tenants and pending migrations without changing anything.
- `deploy` : canary first (first K tenants, wait, run a health check), then waves with bounded concurrency. Default concurrency is `min(8, poolMax - reserve)` so migrations never starve live traffic.

- Per-tenant `pg_try_advisory_lock` so two runners (or a runner and a provisioner) never touch the same schema.
- `lock_timeout` and `statement_timeout` per tenant, retry with backoff on lock timeouts, and a `--max-failures` threshold that stops the run.
- Resumable: per-tenant state `pending → running → applied | failed(error, migration)` stored in the control DB.
- **Version gate.** The registry stores each tenant's schema version. The app declares the minimum version it supports. Policy for tenants behind: reject with a maintenance error, or lazy-migrate on first request under an advisory lock (opt-in).
- **Lint** (`psr migrate lint`): flag DROP COLUMN or TABLE, type changes that rewrite tables, NOT NULL without a default, and non-concurrent index creation. Encourage expand/contract so the app tolerates versions N and N-1 during a rollout.
- **Drift check:** compare each schema's catalog fingerprint (tables, columns, indexes, constraints) to the template and report differences.
- **Rollback:** Postgres DDL is transactional, so each tenant's migration is atomic. Fleet-level rollback means forward-fixing. Document this clearly.

6.4 Provisioning and lifecycle

Strategies

- `replay` (default, most correct): create the schema and apply all migrations. Slower as history grows.
- `baseline`: apply a squashed baseline migration, then later ones.
- `clone-template` (fast, your current approach): copy from a template schema. Cloning must cover tables, indexes, constraints, foreign keys, identity and sequences, enums, functions, triggers and grants. Silent omissions are the risk, so always run the drift check after a clone and fail provisioning on mismatch.

State machine: `provisioning → ready → suspended → archived → deleted`. Creation runs in one transaction (schema plus DDL), so failures roll back cleanly, and provisioning is idempotent per tenant id so retries are safe. Seed hooks run after `ready`.

Deprovisioning: suspend first, then optionally archive with `pg_dump -n` to object storage, hold for a retention window, and only then drop. Never run `DROP SCHEMA ... CASCADE` unless status is `archived` and a confirmation token is supplied.

6.5 Pool governor (D4)

- Wrap Prisma operations with a client extension (`$extends` query hook) that acquires a per-tenant slot before running.
- Per-tenant concurrency cap (default 4), fair round-robin queue across tenants, queue timeout that raises `TenantBusyError`.
- Interactive transactions hold a connection for their whole lifetime, so count a transaction as one slot for its duration.
- Apply `statement_timeout` per transaction with `SET LOCAL`.
- Expose wait time and queue depth per tenant for noisy-neighbor detection.

6.6 Jobs and adapters

- **Job envelope:** `{ tenant: { id, sig }, payload }`. Pass the tenant id explicitly in the message, not through ambient context. Optionally sign it with an HMAC so another service cannot spoof a tenant.
- `router.wrapJob(handler)` rejects any job without a valid tenant claim (route to a dead-letter queue) instead of falling back to `public`.
- `router.forEachTenant(fn, { concurrency })` for cron-style work across all tenants, with each run isolated and bounded.
- Adapters: Express, Fastify, NestJS interceptor, Next.js route-handler wrapper, SQS consumer, BullMQ worker. Your existing SQS code becomes one adapter on a generic `runWithTenant`.

6.7 Observability (D6)

- OpenTelemetry span attribute for tenant (with an option to hash the id).
- Metrics: pool wait time, active and idle connections, client cache size, hits and evictions, governor queue depth, migration lag per tenant, context errors.
- A debug snapshot function returning pool stats and cache contents for health endpoints.

6.8 Isolation conformance suite (D3, the headline feature)

`psr verify` (and a programmatic `verifyIsolation()` for test runners):

1. Read Prisma's model metadata to enumerate every model.
2. Provision ephemeral tenants A, B and C.

3. Seed tenant-stamped canary rows (auto-generated minimal rows, with a factory hook for models that need valid relations).
4. Run probes:
   - Sequential cross-reads: every model queried in each tenant sees only its own canary.
   - Concurrency: 2,000 interleaved tasks across tenants via `Promise.all`, each asserting only its own data is visible.
   - Context-loss fuzzing through `setTimeout`, `setImmediate`, EventEmitter handlers, streams, promisified callbacks and `AsyncResource` boundaries.
   - Raw SQL probes: unqualified queries must error under the guard.
   - Jobs: a job without a tenant claim must throw.
   - Lifecycle: suspended tenants are rejected; after client eviction and reuse the schema is still correct; pool error recovery works.
5. Output a JSON and Markdown report with per-probe pass or fail and a CI exit code.

Be honest in the docs: this is strong evidence for the router and the app's wiring, not a proof about every query an application might write.

7. Public API sketch

```
import { createTenantRouter, pgRegistry } from '@psr/core'

const router = createTenantRouter({
  pool,                                    // one shared pg.Pool
  registry: pgRegistry(controlPool),       // tenant -> { schema, status,
schemaVersion, placement }
  createClient: (adapter) => new PrismaClient({ adapter }),
  cache: { maxClients: 200, idleTtlMs: 10 * 60_000 },
  guard: { mode: 'strict' },
  governor: { perTenantConcurrency: 4, queueTimeoutMs: 5_000 },
  minSchemaVersion: '20261001000000_add_invoices',
})

app.use(router.middleware.express({ resolve: fromSubdomain }))

const db = router.db()                     // throws TenantContextError
outside a tenant context
await router.run(tenantId, async () => { /* tenant-scoped work */ })
await router.system(async (control) => { /* explicit control-plane access,
logged */ })
await router.raw(sql`select count(*) from invoices`)   // tx + SET LOCAL
search_path
await router.forEachTenant(job, { concurrency: 4 })
```

CLI

```
psr migrate plan
psr migrate deploy --concurrency 8 --canary 2 --max-failures 3
psr migrate status
psr migrate resume
psr migrate lint
psr tenants provision acme --strategy replay
psr tenants suspend acme
psr verify
```

8. Control-plane data model

```
create table psr_tenants (
  id uuid primary key,
  slug text unique not null,
  schema_name text unique not null check (schema_name ~ '^t_[a-z0-9_]
{3,40}$'),
  status text not null,                -- provisioning | ready | migrating |
suspended | archived
  schema_version text,
  placement text not null default 'shared',
  created_at timestamptz not null default now()
);

create table psr_migration_runs (
  id uuid primary key,
  started_at timestamptz not null default now(),
  target_version text not null,
  status text not null                 -- running | stopped | completed
);

create table psr_tenant_migrations (
  run_id uuid references psr_migration_runs(id),
  tenant_id uuid references psr_tenants(id),
  state text not null,                 -- pending | running | applied |
failed
  error text,
  updated_at timestamptz not null default now(),
  primary key (run_id, tenant_id)
);
```

9. Testing and quality bar

- Real Postgres in CI (Testcontainers), plus a **PgBouncer in transaction mode** job, since pooler behavior is where schema routing breaks in production.
- CI matrix across supported Prisma versions and Node LTS versions.
- Property and concurrency tests for the client cache (single-flight, eviction during use, reuse after eviction).
- A listener-count regression test and a soak test that switches across 1,000 tenants for an hour and watches memory.

- Failure injection: kill connections mid-migration, drop the pool, make provisioning fail halfway.
- Release hygiene: semantic versioning with changesets, npm provenance, a security policy and a public threat model.

10. Benchmarks to publish

Scenario	Metric
10 / 100 / 1,000 / 5,000 tenants	Open DB connections, process memory, memory per cached client
Router vs a single plain client	p50 and p95 latency overhead
Router vs naive client-per-tenant pools	Connections and memory at the same tenant counts
Router vs shared schema plus RLS (`set_config` in a transaction)	Latency and throughput, so users can choose with data
Fleet migration over 1,000 schemas at concurrency 4, 8, 16	Total time, failure behavior, impact on live query latency
One noisy tenant against 50 quiet ones	p95 for quiet tenants with and without the governor

Publish the scripts so anyone can reproduce the numbers.

11. Roadmap with exit criteria

Phase	Scope	Exit criteria
0. Extract and harden	Pull `tenantDb`, `controlDb`, `resolveTenant` into a framework-free core; remove app-specific code; fix listener root cause	Core passes its own tests with no dependency on your SaaS app

Phase	Scope	Exit criteria
1. v0.1	Router, LRU cache, guard L1 to L3, replay provisioning, `psr verify` v1, Express and Fastify adapters, README, example app	A new user reaches a verified tenant in about 10 minutes
2. v0.2	Fleet migrations v1 (shell out to Prisma), state tables, lint, drift check, CLI	1,000-schema migration completes, resumes after a forced crash, and canary stops a bad migration
3. v0.3	Governor, OTel, `wrapJob` with signed claims, SQS and BullMQ adapters, `forEachTenant`	Noisy-neighbor benchmark shows governor benefit
4. v0.4	Baseline and clone-template strategies, lazy migration, NestJS and Next.js adapters, published benchmarks	Benchmarks reproducible from the repo
5. v1.0	API freeze, optional native migration applier, tiered placement (dedicated pool), threat model and security review, Prisma version matrix	Two external projects running it in production

Size this as a part-time solo project: phases 0 and 1 are the credibility milestone, phase 2 is the engineering centerpiece.

12. Risks and mitigations

Risk	Mitigation
Prisma changes its adapter or schema handling	Keep the Prisma-facing surface tiny, run a version matrix in CI, pin tested ranges
Pooler incompatibility (transaction-mode poolers, prepared statements)	PgBouncer CI job; use `SET LOCAL` only; document supported setups

Risk	Mitigation
Postgres catalog bloat with thousands of schemas times many tables slows migrations, dumps and maintenance	Measure it in benchmarks, document a practical ceiling from your own data, recommend dedicated placement beyond it
AsyncLocalStorage context loss in third-party callbacks	Conformance fuzzing, explicit `runWithTenant` for known-bad libraries, fail closed
Overclaiming security	Use “fail-closed” and “verified by suite”, never “zero-leak”; publish a threat model
Scope creep and maintenance burden	Core stays small; features live in separate packages; clear non-goals
Prisma or another vendor ships something similar	Compete on the operations layer (migrations, verification, governance), which is harder to absorb than routing

13. README rules (claims discipline)

- Lead with the problem and a 20-line example, then the architecture diagram.
- State what it does not do and when to prefer shared schema plus RLS.
- Every performance or safety claim links to a benchmark script or a verify report.
- Do not state popularity claims (“most requested feature”) without linking the issues.
- Include a threat model section: what the guard covers and what remains the application’s job.

14. Open questions

1. How do you run migrations across tenant schemas today? This decides how much of Engine v1 you can reuse.
2. Which Prisma and Node versions run in production, and which pooler or provider (Supabase, RDS, PgBouncer)?
3. Current and target tenant counts, and how many tables per schema?
4. Do tenants share any reference tables in `public`, and how should the guard treat them?

5. Which license do you want (MIT or Apache-2.0), and is a hosted or paid layer ever planned?